Deep Dive — Module A

FastAPI — Complete Documentation

Every feature, every pattern, from HTTP to production deployment

ContentAutomation2 — Backend Deep-Dive Series
Complete Documentation | Zero Prerequisites to Production Code

Table of Contents

1. How HTTP Works — The Foundation

HTTP (HyperText Transfer Protocol) is the language of the web. Every time your browser loads a page, your React app fetches data, or your mobile app logs in, it sends an HTTP **request** and receives an HTTP **response**.

1.1 Anatomy of an HTTP Request

```
POST /api/users HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer eyJhbGc...
Content-Length: 42

{"name": "Alice", "email": "alice@example.com"}

^---- Method  ^---- Path  ^---- Protocol version
              ^---- Headers (key: value pairs)

              ^---- Body (the data you're sending)
```

The **method** tells the server what action you want:

Method	Meaning	Has Body?	Example
GET	Read data	No	GET /users/42
POST	Create new resource	Yes	POST /users
PUT	Replace entire resource	Yes	PUT /users/42
PATCH	Partially update resource	Yes	PATCH /users/42
DELETE	Remove resource	Usually no	DELETE /users/42

1.2 Anatomy of an HTTP Response

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/users/123

{"id": 123, "name": "Alice", "email": "alice@example.com"}

^---- Status code (201 = Created)
     ^---- Headers
           ^---- Body (JSON data)
```

Common status codes:

Code	Meaning	When to use
200 OK	Success	GET, PUT, PATCH succeeded
201 Created	Resource created	POST succeeded
204 No Content	Success, no body	DELETE succeeded
400 Bad Request	Client sent bad data	Validation error, missing field
401 Unauthorized	Not authenticated	No/invalid token
403 Forbidden	Authenticated but no permission	Wrong role

404 Not Found	Resource missing	ID doesn't exist
422 Unprocessable	Validation failed	FastAPI default for body errors
429 Too Many Req	Rate limited	Slow down
500 Server Error	Your code crashed	Unexpected exception

1.3 ASGI vs WSGI — Why FastAPI is Faster

Old frameworks (Django, Flask) use **WSGI** — one thread per request. FastAPI uses **ASGI** — one process can handle thousands of requests concurrently using asyncio. For I/O-bound work (database calls, external APIs), ASGI is drastically more efficient.

---

2. FastAPI vs Flask vs Django

Feature	FastAPI	Flask	Django
Type	ASGI, async-first	WSGI, sync	WSGI, sync (async support added)
Auto docs	Yes (Swagger + ReDoc)	No	No
Data validation	Built-in (Pydantic)	Manual	Forms only
ORM	None (use SQLAlchemy)	None	Django ORM (built-in)
Admin panel	No	No	Yes (built-in)
Learning curve	Low-medium	Low	High
Best for	APIs, microservices, ML	Simple APIs, prototypes	Full-stack web apps

> ***Note:*** *For a pure REST API backend (like ContentAutomation2), FastAPI is the clear winner: fastest, auto-documentation, type safety, and async support out of the box.*

---

3. Installation & Project Structure

```
# Create and activate virtual environment
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate         # macOS/Linux

# Install FastAPI + Uvicorn (ASGI server)
pip install fastapi uvicorn[standard]

# For production installs, also add:
pip install pydantic[email] python-multipart httpx
```

Recommended project layout for a medium-sized API:

```
myapi/
  app/
    __init__.py
    main.py           # FastAPI instance, lifespan, include routers
    config.py         # settings (pydantic-settings)
    deps.py           # shared dependencies (get_db, verify_token)
    models.py         # Pydantic request/response models
    db/
      __init__.py
      client.py       # database connection
      models.py       # ORM models (if using SQLAlchemy)
```

```
    routers/
      __init__.py
      users.py        # /users/* endpoints
      items.py        # /items/* endpoints
      auth.py         # /auth/* endpoints
  tests/
    test_users.py
    test_items.py
  .env
  requirements.txt
  Dockerfile
```

---

4. Your First App & Running the Server

```
# app/main.py
from fastapi import FastAPI

app = FastAPI(
    title="My API",
    description="A sample FastAPI application",
    version="1.0.0",
)

@app.get("/")
def root():
    return {"message": "Hello, World!"}

@app.get("/health")
def health_check():
    return {"status": "ok"}

# Run for development (auto-reloads on file changes):
uvicorn app.main:app --reload --port 8000

# Run for production:
uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4

# With Gunicorn (for production, multi-process):
gunicorn app.main:app -k uvicorn.workers.UvicornWorker -w 4 -b 0.0.0.0:8000
```

Once running, visit:

- http://localhost:8000 — your API
- http://localhost:8000/docs — Swagger UI (interactive docs)
- http://localhost:8000/redoc — ReDoc (cleaner docs)
- http://localhost:8000/openapi.json — raw OpenAPI schema

---

5. Path Parameters

Path parameters are parts of the URL path that vary per request. FastAPI reads the type annotation to validate and convert automatically.

```
from fastapi import FastAPI
from enum import Enum

app = FastAPI()
```

```
# Basic path parameter — auto-converted to int
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}  # user_id is already an int here
# GET /users/42    → {"user_id": 42}
# GET /users/abc   → 422 Validation Error (abc is not an int)

# String path parameter
@app.get("/files/{file_path:path}")  # :path captures slashes too
def get_file(file_path: str):
    return {"file_path": file_path}
# GET /files/images/photo.jpg → {"file_path": "images/photo.jpg"}

# Enum path parameter — constrains valid values
class Status(str, Enum):
    active   = "active"
    inactive = "inactive"
    all      = "all"

@app.get("/users/status/{status}")
def users_by_status(status: Status):
    return {"status": status.value}
# GET /users/status/active   → OK
# GET /users/status/deleted  → 422 (not in enum)

# Multiple path parameters
@app.get("/orgs/{org_id}/repos/{repo_id}")
def get_repo(org_id: int, repo_id: int):
    return {"org": org_id, "repo": repo_id}
```

> ***Important:*** *Order matters: FastAPI matches routes top to bottom. If you have both /users/me and /users/{user_id}, put /users/me FIRST — otherwise 'me' gets matched as a user_id string.*

---

6. Query Parameters

Query parameters appear after ? in the URL. Any function parameter that is NOT a path parameter is treated as a query parameter.

```
from fastapi import FastAPI, Query
from typing import Optional

app = FastAPI()

# Basic query parameters
@app.get("/items")
def list_items(
    skip: int = 0,          # ?skip=10
    limit: int = 20,        # ?limit=5
    q: Optional[str] = None # ?q=hello  (optional)
):
    return {"skip": skip, "limit": limit, "query": q}
# GET /items?skip=5&limit=10&q=laptop

# Query with validation using Query()
@app.get("/products")
def list_products(
    min_price: float = Query(default=0.0, ge=0.0),
    max_price: float = Query(default=10000.0, le=100000.0),
```

```
    category: str    = Query(default="all", min_length=2, max_length=50),
    tags: list[str]  = Query(default=[]),  # ?tags=a&tags=b → ["a", "b"]
):
    return {"min": min_price, "max": max_price, "category": category, "tags": tags}

# Required query parameter (no default)
@app.get("/search")
def search(q: str):   # required — 422 if missing
    return {"results": [q]}

# Deprecated parameter (shows in docs but warns users)
@app.get("/old-search")
def old_search(search: str = Query(default=None, deprecated=True)):
    return {"results": [search]}
```

---

7. Request Body with Pydantic

When you need to receive structured data (JSON), define a Pydantic model and use it as a type annotation. FastAPI automatically reads the request body, validates it, and gives you a populated model instance.

```
from fastapi import FastAPI
from pydantic import BaseModel, EmailStr, Field
from typing import Optional
from datetime import datetime

app = FastAPI()

# ■■ Request model (what the client sends) ■■
class UserCreate(BaseModel):
    name:     str          = Field(..., min_length=2, max_length=100)
    email:    EmailStr                     # validated email format
    age:      int          = Field(..., ge=18, le=120)
    bio:      Optional[str] = None          # optional field

# ■■ Response model (what we return) ■■
class UserResponse(BaseModel):
    id:         int
    name:       str
    email:      str
    created_at: datetime

@app.post("/users", response_model=UserResponse, status_code=201)
def create_user(user: UserCreate):
    # user.name, user.email, user.age are all validated and typed
    new_user = save_to_db(user)     # your DB logic
    return new_user                 # serialised using UserResponse

# ■■ Combining path params + body ■■
@app.put("/users/{user_id}")
def update_user(user_id: int, user: UserCreate):
    # user_id from path, user from body
    return {"user_id": user_id, "updated": user.name}

# ■■ Multiple bodies (less common) ■■
class Item(BaseModel):
    name: str
    price: float
```

```
class Order(BaseModel):
    item: Item
    quantity: int

@app.post("/orders")
def create_order(order: Order):
    return {"total": order.item.price * order.quantity}
```

7.1 Nested Models

```
from pydantic import BaseModel
from typing import list

class Address(BaseModel):
    street: str
    city:   str
    pin:    str

class Company(BaseModel):
    name:    str
    address: Address     # nested model
    tags:    list[str]   # list of strings

@app.post("/companies")
def create_company(company: Company):
    # company.address.city  — nested access works naturally
    return company.model_dump()

# JSON body expected:
# {
#   "name": "Growthvine Capital",
#   "address": {"street": "MG Road", "city": "Mumbai", "pin": "400001"},
#   "tags": ["finance", "amfi", "sebi"]
# }
```

7.2 Body with Extra Fields via Field()

```
from pydantic import BaseModel, Field
from typing import Optional
import uuid

class IdeaCreate(BaseModel):
    # Required fields
    angle:    str   = Field(..., min_length=10, description="Content angle")
    platform: str   = Field(..., pattern="^(linkedin|twitter|blog|email)$")

    # Optional with defaults
    score:    float = Field(default=5.0, ge=0.0, le=10.0)
    tags:     list[str] = Field(default_factory=list)

    # Alias: JSON field name differs from Python attribute name
    agent_id: str   = Field(default_factory=lambda: str(uuid.uuid4()),
                            alias="agentId")

    class Config:
        populate_by_name = True   # allow both 'agent_id' and 'agentId'
```

---

8. Response Models & Status Codes

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import Optional

app = FastAPI()

class UserDB(BaseModel):     # what's in the database
    id:             int
    name:           str
    email:          str
    hashed_password: str     # SECRET — should not be returned

class UserOut(BaseModel):    # what we return to the client
    id:    int
    name:  str
    email: str
    # hashed_password omitted — the response_model filters it out!

@app.get("/users/{user_id}", response_model=UserOut, status_code=200)
def get_user(user_id: int) -> UserDB:
    user = fetch_from_db(user_id)   # returns UserDB (with password)
    return user                     # FastAPI filters to UserOut fields

# ■■ Multiple response types ■■
from typing import Union

class AdminOut(UserOut):
    role: str

@app.get("/users/{user_id}/full",
         response_model=Union[AdminOut, UserOut])
def get_user_full(user_id: int):
    ...

# ■■ response_model_exclude_none ■■
@app.get("/items/{item_id}",
         response_model=ItemOut,
         response_model_exclude_none=True)  # omit None fields from response
def get_item(item_id: int):
    ...

# ■■ Return a plain dict when no model needed ■■
@app.get("/ping")
def ping():
    return {"status": "ok", "timestamp": "2026-06-03"}
```

---

9. Headers, Cookies & Form Data

```
from fastapi import FastAPI, Header, Cookie
from typing import Optional

app = FastAPI()

# ■■ Reading headers ■■
@app.get("/items")
def get_items(
```

```
    user_agent: Optional[str] = Header(default=None),
    x_request_id: Optional[str] = Header(default=None),
    # Header names are auto-converted: x_request_id → X-Request-Id
):
    return {"user_agent": user_agent, "request_id": x_request_id}

# ■■ Reading cookies ■■
@app.get("/me")
def get_me(session_token: Optional[str] = Cookie(default=None)):
    return {"session": session_token}

# ■■ Setting cookies in response ■■
from fastapi.responses import JSONResponse

@app.post("/login")
def login(response: JSONResponse):
    response.set_cookie(key="session_token", value="abc123",
                        httponly=True, max_age=3600)
    return {"status": "logged in"}

# ■■ Form data (HTML forms) ■■
# pip install python-multipart
from fastapi import Form

@app.post("/login-form")
def login_form(username: str = Form(...), password: str = Form(...)):
    return {"username": username}
```

---

10. File Uploads

```
from fastapi import FastAPI, File, UploadFile
from typing import list

app = FastAPI()

# ■■ Single file upload ■■
@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    content = await file.read()   # read file bytes
    return {
        "filename":     file.filename,
        "content_type": file.content_type,
        "size_bytes":   len(content),
    }

# ■■ Multiple files ■■
@app.post("/upload/multiple")
async def upload_multiple(files: list[UploadFile] = File(...)):
    results = []
    for file in files:
        content = await file.read()
        results.append({"name": file.filename, "size": len(content)})
    return results

# ■■ Save file to disk ■■
import shutil
from pathlib import Path

UPLOAD_DIR = Path("uploads")
```

```
UPLOAD_DIR.mkdir(exist_ok=True)

@app.post("/upload/save")
async def save_file(file: UploadFile = File(...)):
    dest = UPLOAD_DIR / file.filename
    with dest.open("wb") as f:
        shutil.copyfileobj(file.file, f)   # stream — memory efficient
    return {"saved": str(dest)}

# ■■ File + form fields together ■■
@app.post("/upload/with-metadata")
async def upload_with_metadata(
    file:        UploadFile = File(...),
    description: str        = Form(...),
    tags:        str        = Form(default=""),
):
    return {"file": file.filename, "desc": description}
```

---

11. Error Handling & HTTPException

```
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import JSONResponse
from fastapi.exception_handlers import http_exception_handler

app = FastAPI()

# ■■ Basic HTTPException ■■
@app.get("/users/{user_id}")
def get_user(user_id: int):
    user = db.get(user_id)
    if not user:
        raise HTTPException(
            status_code=404,
            detail=f"User {user_id} not found",
            headers={"X-Error": "not-found"},   # optional custom headers
        )
    return user

# ■■ Custom exception class ■■
class InsufficientFunds(Exception):
    def __init__(self, balance: float, required: float):
        self.balance  = balance
        self.required = required

# ■■ Custom exception handler ■■
@app.exception_handler(InsufficientFunds)
async def insufficient_funds_handler(request: Request, exc: InsufficientFunds):
    return JSONResponse(
        status_code=402,
        content={
            "error": "insufficient_funds",
            "balance":  exc.balance,
            "required": exc.required,
        }
    )

@app.post("/pay")
def pay(amount: float):
    balance = get_balance()
```

```
    if balance < amount:
        raise InsufficientFunds(balance=balance, required=amount)
    return {"paid": amount}

# ■■ Override FastAPI's default 422 validation error response ■■
from fastapi.exceptions import RequestValidationError

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request: Request, exc: RequestValidationError):
    return JSONResponse(
        status_code=400,   # change 422 → 400 if you prefer
        content={"errors": exc.errors(), "body": str(exc.body)},
    )
```

---

12. Dependency Injection (Depends)

Dependencies are reusable functions (or classes) that FastAPI calls automatically before your route handler. They are great for: authentication, database sessions, pagination parameters, and any shared logic.

```
from fastapi import FastAPI, Depends, HTTPException, Header
from typing import Optional

app = FastAPI()

# ■■ Simple dependency ■■
def get_pagination(skip: int = 0, limit: int = 20):
    # This function runs before the route handler
    # Its return value is injected as the 'pagination' argument
    return {"skip": skip, "limit": limit}

@app.get("/items")
def list_items(pagination: dict = Depends(get_pagination)):
    # pagination = {"skip": 0, "limit": 20} (from query params)
    return {"items": [], **pagination}

# ■■ Auth dependency ■■
async def verify_api_key(x_api_key: str = Header(...)):
    if x_api_key != "secret-123":
        raise HTTPException(status_code=401, detail="Invalid API key")
    return x_api_key   # returned value available if the route needs it

@app.get("/protected")
def protected(api_key: str = Depends(verify_api_key)):
    return {"message": "access granted", "key": api_key}

# ■■ Dependency with yield — for DB sessions ■■
def get_db():
    db = SessionLocal()   # open DB session
    try:
        yield db          # inject into route handler
    finally:
        db.close()        # always close, even on error

@app.get("/users/{user_id}")
def get_user(user_id: int, db = Depends(get_db)):
    return db.query(User).filter(User.id == user_id).first()
```

12.1 Applying Dependencies to All Routes in a Router

```
from fastapi import APIRouter, Depends
from app.deps import verify_api_key

# Apply auth to every route in this router
router = APIRouter(
    prefix="/admin",
    tags=["admin"],
    dependencies=[Depends(verify_api_key)],
)

@router.get("/stats")         # auth applied automatically
def admin_stats(): ...

@router.delete("/users/{id}") # auth applied automatically
def delete_user(id: int): ...
```

---

13. Nested Dependencies & Class-Based Dependencies

```
from fastapi import Depends

# ■■ Nested dependency (dep A depends on dep B) ■■
def get_settings():
    return {"api_url": "https://api.example.com", "timeout": 30}

def get_http_client(settings: dict = Depends(get_settings)):
    return httpx.Client(base_url=settings["api_url"],
                        timeout=settings["timeout"])

@app.get("/data")
def get_data(client = Depends(get_http_client)):
    return client.get("/resource").json()
# FastAPI automatically resolves: get_settings → get_http_client → get_data

# ■■ Class-based dependency (callable class) ■■
class Pagination:
    def __init__(self, page: int = 1, page_size: int = 20, max_page_size: int = 100):
        if page_size > max_page_size:
            raise HTTPException(400, detail=f"page_size cannot exceed {max_page_size}")
        self.skip  = (page - 1) * page_size
        self.limit = page_size
        self.page  = page

@app.get("/products")
def list_products(pagination: Pagination = Depends(Pagination)):
    # pagination.skip, pagination.limit, pagination.page all available
    return {"skip": pagination.skip, "limit": pagination.limit}

# ■■ Security dependency with scopes ■■
from fastapi.security import SecurityScopes

class RoleChecker:
    def __init__(self, allowed_roles: list[str]):
        self.allowed_roles = allowed_roles

    def __call__(self, user = Depends(get_current_user)):
        if user.role not in self.allowed_roles:
            raise HTTPException(403, detail="Insufficient permissions")
```

```
require_admin = RoleChecker(allowed_roles=["admin", "superuser"])
require_user  = RoleChecker(allowed_roles=["admin", "superuser", "user"])

@app.delete("/users/{id}", dependencies=[Depends(require_admin)])
def delete_user(id: int): ...
```

---

14. APIRouter — Splitting Your App into Modules

```
# routers/users.py
from fastapi import APIRouter, Depends, HTTPException
from app.deps import verify_api_key, get_db
from app.models import UserCreate, UserOut

router = APIRouter(
    prefix="/users",          # all routes start with /users
    tags=["Users"],           # grouping in Swagger docs
    dependencies=[Depends(verify_api_key)],  # auth for all routes
)

@router.get("/", response_model=list[UserOut])
def list_users(db = Depends(get_db)):
    return db.query(User).all()

@router.get("/{user_id}", response_model=UserOut)
def get_user(user_id: int, db = Depends(get_db)):
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(404, "User not found")
    return user

@router.post("/", response_model=UserOut, status_code=201)
def create_user(user: UserCreate, db = Depends(get_db)):
    db_user = User(**user.model_dump())
    db.add(db_user)
    db.commit()
    return db_user

@router.delete("/{user_id}", status_code=204)
def delete_user(user_id: int, db = Depends(get_db)):
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(404, "User not found")
    db.delete(user)
    db.commit()

# app/main.py — include all routers
from fastapi import FastAPI
from app.routers import users, items, auth, admin

app = FastAPI()

app.include_router(auth.router)                  # /auth/*
app.include_router(users.router)                 # /users/*
app.include_router(items.router, prefix="/api")  # /api/items/*
app.include_router(admin.router,
    prefix="/admin",
    tags=["Admin"],
    dependencies=[Depends(require_admin)],
)
```

15. Middleware

Middleware runs on EVERY request and response. Common uses: logging, request ID injection, rate limiting, response time headers.

```
from fastapi import FastAPI, Request
from fastapi.middleware.base import BaseHTTPMiddleware
import time, uuid

app = FastAPI()

# ■■ Method 1: @app.middleware decorator ■■
@app.middleware("http")
async def add_request_id(request: Request, call_next):
    request_id = str(uuid.uuid4())[:8]
    request.state.request_id = request_id

    start = time.time()
    response = await call_next(request)   # run the actual route
    duration = time.time() - start

    response.headers["X-Request-Id"]    = request_id
    response.headers["X-Response-Time"] = f"{duration:.3f}s"
    return response

# ■■ Method 2: Class-based middleware ■■
class RateLimitMiddleware(BaseHTTPMiddleware):
    def __init__(self, app, max_requests: int = 100):
        super().__init__(app)
        self.max_requests = max_requests
        self._counts: dict[str, int] = {}

    async def dispatch(self, request: Request, call_next):
        ip = request.client.host
        self._counts[ip] = self._counts.get(ip, 0) + 1
        if self._counts[ip] > self.max_requests:
            from fastapi.responses import JSONResponse
            return JSONResponse({"error": "rate limited"}, status_code=429)
        return await call_next(request)

app.add_middleware(RateLimitMiddleware, max_requests=100)
```

16. CORS — Cross-Origin Resource Sharing

Browsers block cross-origin requests by default. CORS headers tell the browser which origins are allowed to talk to your API.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

# ■■ Development (permissive) ■■
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000", "http://localhost:5173"],
```

```
    allow_credentials=True,     # allow cookies to be sent
    allow_methods=["*"],        # GET, POST, PUT, DELETE, etc.
    allow_headers=["*"],        # Authorization, Content-Type, etc.
)

# ■■ Production (strict) ■■
app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://yourdomain.com", "https://app.yourdomain.com"],
    allow_credentials=False,
    allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE"],
    allow_headers=["Authorization", "Content-Type", "X-API-Key"],
    max_age=600,    # browser can cache preflight for 10 minutes
)
```

Important: *allow_origins=['*'] with allow_credentials=True is INVALID. Either specify exact origins with credentials, or use ['*'] without credentials.*

---

17. Background Tasks

Background tasks run after the response is sent — useful for sending emails, triggering webhook notifications, or logging without delaying the response.

```
from fastapi import FastAPI, BackgroundTasks

app = FastAPI()

def send_welcome_email(email: str, name: str):
    # Runs AFTER the response is returned
    import smtplib
    print(f"Sending welcome email to {email}")
    # ... actual email logic ...

def log_event(event_type: str, data: dict):
    print(f"Logging: {event_type} — {data}")

@app.post("/users", status_code=201)
async def create_user(user: UserCreate, background_tasks: BackgroundTasks):
    new_user = save_user_to_db(user)    # synchronous — save first

    # Schedule tasks to run after response is sent
    background_tasks.add_task(send_welcome_email, user.email, user.name)
    background_tasks.add_task(log_event, "user_created", {"id": new_user.id})

    return new_user   # response sent immediately
```

Note: *BackgroundTasks is simple and synchronous. For heavy, long-running, or reliable background work, use a proper job queue like arq + Redis (see Module 03 in this series).*

---

18. WebSockets

WebSockets provide full-duplex (two-way) communication over a single connection. Used for real-time features: chat, live notifications, streaming AI responses.

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect

app = FastAPI()
```

```python
# ■■ Simple echo WebSocket ■■
@app.websocket("/ws/echo")
async def echo(websocket: WebSocket):
    await websocket.accept()
    try:
        while True:
            data = await websocket.receive_text()
            await websocket.send_text(f"Echo: {data}")
    except WebSocketDisconnect:
        print("Client disconnected")

# ■■ WebSocket with connection manager ■■
class ConnectionManager:
    def __init__(self):
        self.active: list[WebSocket] = []

    async def connect(self, ws: WebSocket):
        await ws.accept()
        self.active.append(ws)

    def disconnect(self, ws: WebSocket):
        self.active.remove(ws)

    async def broadcast(self, message: str):
        for ws in self.active:
            await ws.send_text(message)

manager = ConnectionManager()

@app.websocket("/ws/chat/{room}")
async def chat(websocket: WebSocket, room: str):
    await manager.connect(websocket)
    try:
        while True:
            msg = await websocket.receive_text()
            await manager.broadcast(f"[{room}] {msg}")
    except WebSocketDisconnect:
        manager.disconnect(websocket)

# ■■ Streaming AI responses (SSE-like pattern) ■■
@app.websocket("/ws/ai")
async def ai_stream(websocket: WebSocket):
    await websocket.accept()
    msg = await websocket.receive_text()
    async for chunk in stream_from_claude(msg):   # your streaming LLM call
        await websocket.send_text(chunk)
    await websocket.send_text("[DONE]")
```

---

19. Security — API Keys, OAuth2, JWT

API Key Authentication

```python
from fastapi import Security, HTTPException
from fastapi.security import APIKeyHeader

api_key_header = APIKeyHeader(name="X-API-Key", auto_error=True)

VALID_KEYS = {"secret-key-123", "another-valid-key"}
```

```
async def verify_api_key(api_key: str = Security(api_key_header)):
    if api_key not in VALID_KEYS:
        raise HTTPException(status_code=401, detail="Invalid API key")
    return api_key
```

JWT Authentication (Full Example)

```
pip install python-jose[cryptography] passlib[bcrypt]

from jose import jwt, JWTError
from passlib.context import CryptContext
from datetime import datetime, timedelta, timezone
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm

SECRET_KEY = "your-secret-key-here-keep-this-safe"
ALGORITHM  = "HS256"
TOKEN_EXPIRE_MINUTES = 60

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/token")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)

def create_access_token(data: dict) -> str:
    payload = data.copy()
    payload["exp"] = datetime.now(timezone.utc) + timedelta(minutes=TOKEN_EXPIRE_MINUTES)
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

async def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id: int = payload.get("sub")
        if user_id is None:
            raise HTTPException(401, "Invalid token")
    except JWTError:
        raise HTTPException(401, "Invalid token")
    user = db.get(user_id)
    if not user:
        raise HTTPException(401, "User not found")
    return user

@app.post("/auth/token")
def login(form: OAuth2PasswordRequestForm = Depends()):
    user = db.get_by_email(form.username)
    if not user or not verify_password(form.password, user.hashed_password):
        raise HTTPException(401, "Incorrect credentials")
    token = create_access_token({"sub": str(user.id)})
    return {"access_token": token, "token_type": "bearer"}

@app.get("/me")
def get_me(current_user = Depends(get_current_user)):
    return current_user
```

---

20. Lifespan — Startup & Shutdown

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
from typing import AsyncIterator

@asynccontextmanager
async def lifespan(app: FastAPI) -> AsyncIterator[None]:
    # ■■■■■■■■■■■■■■■■■ STARTUP ■■■■■■■■■■■■■■■■■
    print("Starting up...")

    # Initialize database connection pool
    app.state.db_pool = await asyncpg.create_pool(DATABASE_URL)

    # Connect to Redis
    app.state.redis = await aioredis.create_redis_pool(REDIS_URL)

    # Load ML model into memory (once — not per-request)
    app.state.model = load_my_ml_model()

    # Create arq job pool
    import arq
    from app.queue.worker import WorkerSettings
    app.state.arq_pool = await arq.create_pool(WorkerSettings.redis_settings)

    print("Startup complete")

    yield   # ← application is running here, serving requests

    # ■■■■■■■■■■■■■■■■■ SHUTDOWN ■■■■■■■■■■■■■■■■■
    print("Shutting down...")
    await app.state.arq_pool.close()
    await app.state.redis.close()
    await app.state.db_pool.close()
    print("Shutdown complete")

app = FastAPI(lifespan=lifespan)

# Accessing shared state in a dependency:
def get_db(request: Request):
    return request.app.state.db_pool

def get_arq(request: Request):
    return request.app.state.arq_pool
```

---

21. Custom Responses & Streaming

```
from fastapi import FastAPI
from fastapi.responses import (
    JSONResponse,
    HTMLResponse,
    PlainTextResponse,
    RedirectResponse,
    FileResponse,
    StreamingResponse,
)

app = FastAPI()

# ■■ JSON with custom status ■■
@app.get("/custom")
```

```python
def custom():
    return JSONResponse(
        content={"hello": "world"},
        status_code=202,
        headers={"X-Custom-Header": "value"},
    )

# ■■ HTML response ■■
@app.get("/page", response_class=HTMLResponse)
def page():
    return "<html><body><h1>Hello</h1></body></html>"

# ■■ Redirect ■■
@app.get("/old-url")
def old_url():
    return RedirectResponse(url="/new-url", status_code=301)

# ■■ File download ■■
@app.get("/download/{filename}")
def download(filename: str):
    return FileResponse(
        path=f"files/{filename}",
        media_type="application/pdf",
        filename=filename,        # Content-Disposition: attachment
    )

# ■■ Streaming response (large files / generated data) ■■
import asyncio

async def generate_csv():
    yield "id,name,score\n"
    for i in range(1000):
        await asyncio.sleep(0)   # yield control
        yield f"{i},Item {i},{i * 1.5}\n"

@app.get("/export.csv")
def export():
    return StreamingResponse(
        generate_csv(),
        media_type="text/csv",
        headers={"Content-Disposition": "attachment; filename=export.csv"},
    )

# ■■ Server-Sent Events (SSE) for AI streaming ■■
async def stream_ai_response(prompt: str):
    async for chunk in call_claude_streaming(prompt):
        yield f"data: {chunk}\n\n"   # SSE format
    yield "data: [DONE]\n\n"

@app.get("/ai/stream")
def ai_stream(prompt: str):
    return StreamingResponse(
        stream_ai_response(prompt),
        media_type="text/event-stream",
        headers={"Cache-Control": "no-cache"},
    )
```

---

22. Database Integration

With Supabase (as used in ContentAutomation2)

```
# app/db/client.py
from supabase import create_client, Client
from functools import lru_cache
from app.config import get_settings

@lru_cache(maxsize=1)
def get_supabase_client() -> Client:
    settings = get_settings()
    return create_client(settings.supabase_url, settings.supabase_key)

# app/deps.py — dependency for routes
from fastapi import Depends
from supabase import Client

def get_db() -> Client:
    return get_supabase_client()

# In a route:
@app.get("/ideas")
def list_ideas(db: Client = Depends(get_db)):
    resp = db.table("ideas").select("*").eq("approval_status", "pending_approval").execute()
    return resp.data
```

With SQLAlchemy (traditional ORM approach)

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session

DATABASE_URL = "postgresql://user:pass@localhost/mydb"
engine = create_engine(DATABASE_URL, pool_size=10, max_overflow=20)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

# ■■ ORM model ■■
from sqlalchemy import Column, Integer, String, DateTime
from sqlalchemy.sql import func

class User(Base):
    __tablename__ = "users"
    id         = Column(Integer, primary_key=True)
    name       = Column(String, nullable=False)
    email      = Column(String, unique=True, nullable=False)
    created_at = Column(DateTime, server_default=func.now())

# ■■ Dependency ■■
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# ■■ Route ■■
@app.get("/users/{id}")
def get_user(id: int, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.id == id).first()
    if not user:
        raise HTTPException(404, "User not found")
```

```
    return user
```

---

23. Testing with TestClient

```
pip install httpx pytest pytest-asyncio

# tests/test_users.py
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

# ■■ Basic tests ■■
def test_root():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Hello, World!"}

def test_get_user_not_found():
    response = client.get("/users/9999")
    assert response.status_code == 404
    assert response.json()["detail"] == "User not found"

def test_create_user():
    response = client.post("/users", json={
        "name": "Alice",
        "email": "alice@example.com",
        "age": 25,
    })
    assert response.status_code == 201
    data = response.json()
    assert data["name"] == "Alice"
    assert "id" in data

def test_create_user_invalid_email():
    response = client.post("/users", json={"name": "Bob", "email": "not-an-email", "age": 30})
    assert response.status_code == 422   # validation error

# ■■ With auth header ■■
def test_protected_endpoint():
    response = client.get("/admin", headers={"X-API-Key": "secret-123"})
    assert response.status_code == 200

    response = client.get("/admin", headers={"X-API-Key": "wrong-key"})
    assert response.status_code == 401

# ■■ Override dependencies in tests ■■
def override_get_db():
    db = TestingSessionLocal()  # use a test DB
    try:
        yield db
    finally:
        db.close()

app.dependency_overrides[get_db] = override_get_db

# Run tests: pytest tests/ -v
```

---

24. OpenAPI & Interactive Docs

```
# Customising the docs
app = FastAPI(
    title="ContentAutomation API",
    description=(
        "## Content Automation System\n"
        "- Gate 1: Approve/reject content ideas\n"
        "- Gate 2: Approve/reject drafts\n"
        "- Triggers: Run research, scoring, creation agents"
    ),
    version="0.1.0",
    docs_url="/docs",     # Swagger UI
    redoc_url="/redoc",   # ReDoc
    openapi_url="/openapi.json",
    contact={"name": "Himanshu", "email": "hi@growthvine.in"},
    license_info={"name": "Proprietary"},
)

# Add descriptions to routes
@app.get(
    "/ideas/{idea_id}",
    summary="Get a content idea",
    description="Retrieve a single content idea by its UUID.",
    response_description="The idea with all fields",
    tags=["Ideas"],
    operation_id="get_idea_by_id",
)
def get_idea(idea_id: str):
    ...

# Exclude endpoints from docs (e.g. internal routes)
@app.get("/internal/health", include_in_schema=False)
def internal_health():
    return {"ok": True}
```

---

25. Deployment — Docker + Uvicorn + Gunicorn

```
# Dockerfile
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# For production: Gunicorn manages Uvicorn workers
CMD ["gunicorn", "app.main:app",
     "-k", "uvicorn.workers.UvicornWorker",
     "-w", "4",           # 4 worker processes
     "-b", "0.0.0.0:8000",
     "--timeout", "120",
     "--graceful-timeout", "30"]

# Build and run:
# docker build -t myapi .
```

```
# docker run -p 8000:8000 --env-file .env myapi

# docker-compose.yml — API + worker together
version: "3.9"
services:
  api:
    build: .
    ports: ["8000:8000"]
    env_file: .env
    depends_on: [redis]

  worker:
    build: .
    command: python -m arq app.queue.worker.WorkerSettings
    env_file: .env
    depends_on: [redis]

  redis:
    image: redis:alpine
    ports: ["6379:6379"]
```

---

26. ContentAutomation2 — Full API Walkthrough

Here is how all 7 routers fit together in ContentAutomation2:

```
# All routers and what they do:

GET  /ideas                          → list pending ideas
GET  /ideas/{idea_id}                → get one idea + source article
PATCH /ideas/{idea_id}/approve       → Gate 1: approve with optional angle edit
PATCH /ideas/{idea_id}/reject        → Gate 1: reject + record reason

GET  /drafts                         → list pending drafts
GET  /drafts/{draft_id}              → get full draft + finance flags
PATCH /drafts/{draft_id}/approve     → Gate 2: approve + set scheduled_at
PATCH /drafts/{draft_id}/reject      → Gate 2: reject

POST /triggers/research              → start research agent
POST /triggers/scoring               → start scoring agent
POST /triggers/creation              → start creation agent for idea_ids

GET  /tables/raw-content             → all scraped articles (paginated)
GET  /tables/published-posts         → all published posts
GET  /tables/run-logs                → all agent run logs
GET  /tables/cost-log                → daily token costs
GET  /tables/site-health             → per-site success/failure history
GET  /tables/analytics               → content performance metrics

POST /knowledge-base/upload          → upload PDF/TXT file to KB
GET  /knowledge-base/files           → list KB files

GET  /unsubscribe?token=...          → unsubscribe from email list (public)

WS   /orchestrator/chat              → streaming WebSocket chat with AI
```